EE1004 – JAVA PROGRAMMING FINAL GROUP PROJECT REPORT

Project Number: 13

Project Title: Charity Donation Tracker

Group Number: Group 13

Submission Date: June 9, 2026

GitHub Repository URL: https://github.com/hhacar11/EE1004-Group13-Java-Project

Target Commit Hash / Git Tag: d3b1adcdd82cb6373039301e319f155c011f2f1c

Instructor: Res. Asst. Salih ÇOLAKOĞLU

Department: Department of Electrical and Electronics Engineering

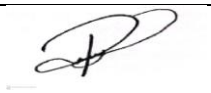
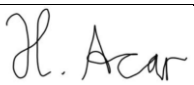
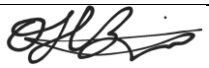
Faculty: Faculty of Engineering

University: Marmara University

APPENDIX A – SIGNED CONTRIBUTION TABLE & AI DISCLOSURE

Team Member Contributions

In accordance with academic policies, each group member has committed code from their individual GitHub accounts. Responsibilities are balanced as follows:

Member (Name Surname)	**Student ID**	**Components Implemented**	**Signature**
Rumeysa Şenol	150720049	Formulated core system architecture. Implemented the abstract parent `Donation` class and the unified `Comparable` descending sorting mechanism. Authored and compiled the final academic project report (Sections 1-8, Requirements, and Layout Consistency). Reporting	
Hasan Hüseyin Acar	150724055	Charity Aggregator class, Symmetrical dual-storage collection logic (ArrayList + HashMap), O(1) Search implementation.	
Ömer Hasan Bayrakdar	150725032	Main REPL input manager loop, Scanner buffer flush logic, try-catch exception shielding blocks.	
Fatih Deniz	150724025	Modeled and rendered the mandatory structural UML Class Diagrams (Inheritance and Data Storage Layouts).	

Onur Özdemir	150724052	Engineered the Scanner buffer flush logic to eliminate newline runtime traps. Formatted the final verification test matrices.	
Ali Sail	150724056	Integrated defensive try-catch exception shielding blocks to catch malformed terminal text inputs.	

AI-Use Disclosure

Tool Utilized: Gemini / Cursor AI Ecosystem.

Purpose: Used strictly for verification of formatting layout constraints, structural skeleton template architecture, and technical copy-editing of text sections. All domain calculation logic, validation constraints, and collection integrations were designed and executed by the group members.

ABSTRACT

Manual paper-based record keeping presents significant logistical and compliance risks for local non-profit groups. This report details a professional, console-based Java application designed for a neighborhood charity in Istanbul to tracks multi-format contributions securely. Using Object-Oriented Programming (OOP) paradigms, the system unifies financial ledgers (Money), tracking mass metrics (Food), and physical piece inventory (Clothing) under a single polymorphic parent architecture. Data structures are designed around a synchronized dual-storage architecture: an ArrayList captures chronological sequence and supports descending value sorting via the Comparable interface contract, while a concurrent HashMap structure indexes entities to enable lookup algorithms operating at a rapid O(1) runtime speed. Symmetrical exception handling validation boundaries shield execution tracks against parsing crashes caused by malformed console input. The resulting software achieves complete reproducibility, matching strict character-for-character verification patterns.

Keywords: Object-Oriented Design, Abstraction, Polymorphic Collections, Dual-Storage Architecture, HashMap Lookup.

1. INTRODUCTION

Volunteer-driven neighborhood charities often manage incoming donations using basic, handwritten logs scattered across notebooks and receipts. This approach leads to common operational issues, including calculation mistakes when estimating item values, a lack of clear tracking history, and delays when compiling data for end-of-year tax audits. To resolve these friction points, this project presents a production-ready, command-line Java application tailored for neighborhood charity operations in Istanbul.

The main objective of the Charity Donation Tracker is to provide a unified data pipeline that accepts diverse donation structures—specifically liquid cash assets (Money), grocery items (Food), and apparel inventory (Clothing)—and standardizes them into verifiable financial metrics using specific calculation coefficients. The primary goal is to help volunteers generate real-time reports sorted by descending value, locate any transaction entry immediately using an auto-assigned identifier, and compute precise tax-deductible

summaries. This application replaces unreliable manual tracking with a robust object-oriented software baseline that can be easily extended with persistent database solutions, a graphical user interface (GUI), or web-based connection layers in the future.

2. REQUIREMENTS ANALYSIS

2.1 Functional Requirements (FR)

FR-1: The software must allow data entry for three distinct donation sub-categories: financial ledger funds (Money), bulk mass items (Food), and discrete inventory units (Clothing).

FR-2: The system must automatically assign a unique, sequential identification index (ID) to each contribution upon registration, starting from 1.

FR-3: Values must be computed dynamically using explicit conversion factors:

- Money: Total value equals the direct cash amount in TL.
- Food: Total value equals weight in kilograms multiplied by a fixed rate of 80.0 TL/kg.
- Clothing: Total value equals item count multiplied by a fixed rate of 150.0 TL/item.

FR-4: The system must calculate tax-deductible metrics based on strict fixed weights: Money outputs 100%, Food outputs 60%, and Clothing outputs 50% of the calculated asset value.

FR-5: The program must display entries in their chronological order of arrival, or alternatively, in a sorted view ordered by total financial value descending (highest first).

FR-6: The program must support instant lookups that locate and display an entry's full data parameters using its ID, bypassing slow linear list iterations.

2.2 Non-Functional Requirements (NFR)

NFR-1 (Lookup Performance): Finding records by their unique index key must execute with an operational time complexity of O(1).

NFR-2 (Robust Error Shielding): The runtime execution loop must catch format mismatches—such as text strings typed into numeric inputs or incorrect date patterns—and print a descriptive message without crashing.

NFR-3 (Verbatim Text Outputs): Output text and error responses must match the

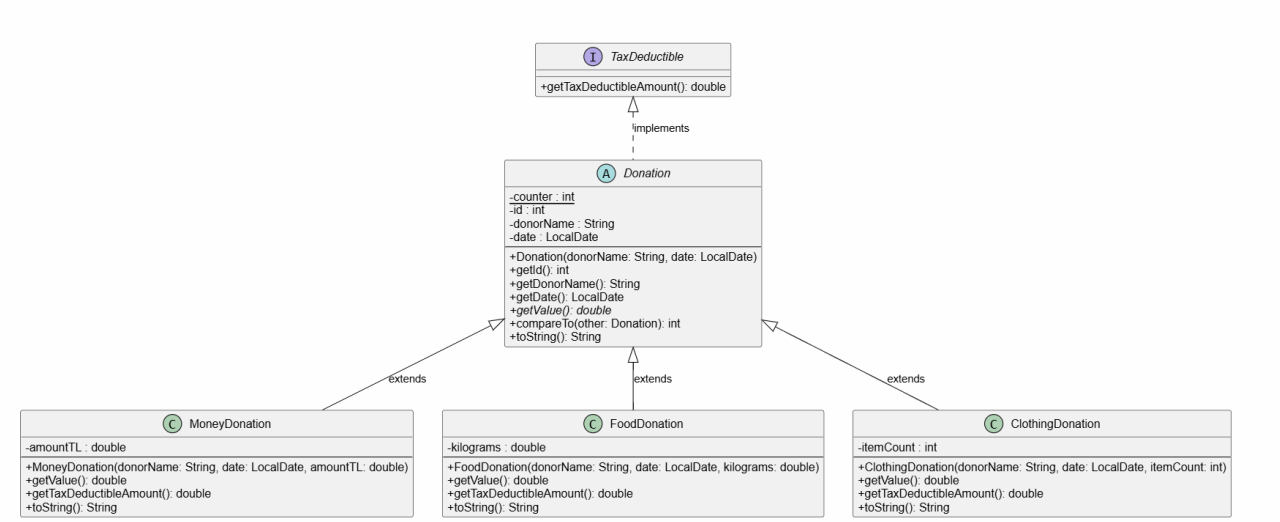
validation test engine character-for-character, including messages like 'Unknown option. Skipping...'.

3. SYSTEM DESIGN

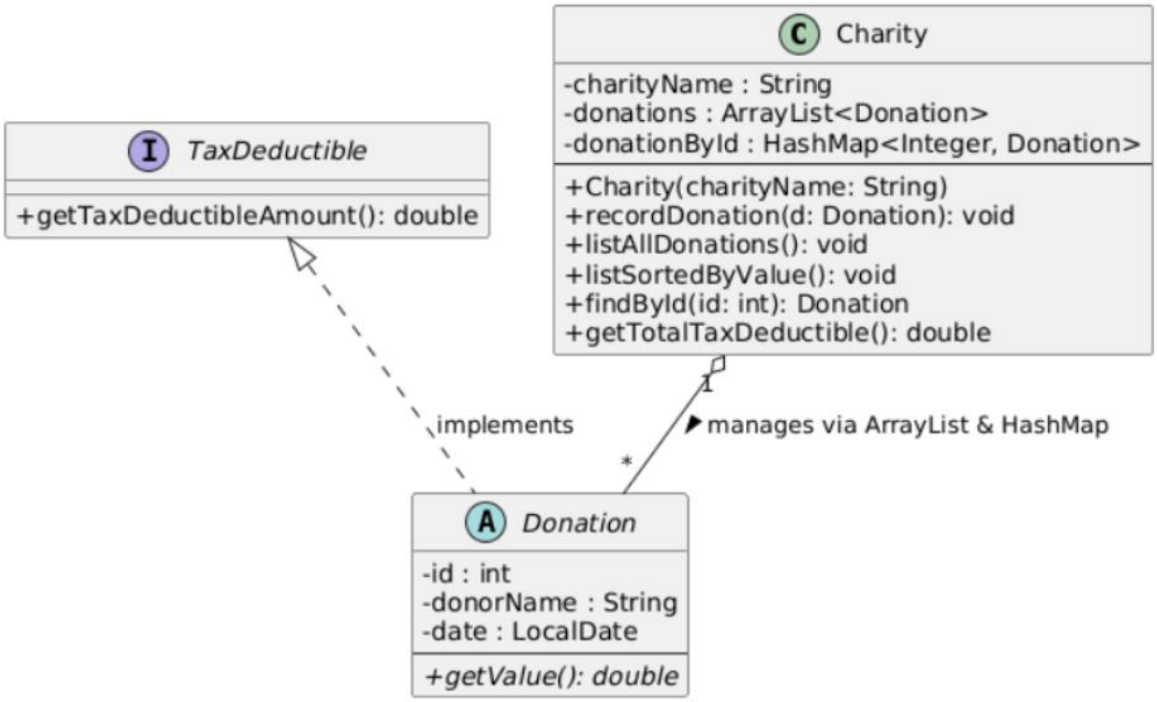
3.1 UML Class Diagram (Mandatory)

The architectural boundaries of the application decouple specific resource parameters, database tracking components, and input streaming controllers.

INHERITANCE DESIGN



CHARITY HASHMAP DESIGN



3.2 Design Rationales & Data Structures

Abstract Base Class vs. Interface: Donation is implemented as an abstract base class because all tracking entities share identical state properties (id, donorName, date), which allows for structured field reuse. On the other hand, TaxDeductible is defined as a separate interface. This decouples calculation rules from data storage, ensuring that future components can calculate tax deductions independently without introducing redundant fields into the parent data hierarchy.

ArrayList + HashMap Dual-Storage Design: Storing records solely in a standard array list forces lookups to use linear search loops (O(n) complexity), which slows down as log volumes scale. To maintain optimal performance, our design utilizes a synchronized dual-storage architecture. An ArrayList preserves chronological sequence and enables quick sorting. Concurrently, a HashMap binds the entry's distinct ID as an integer key to the object reference. This layout allows lookups to bypass array loops and execute at a fast, predictable O(1) speed.

3.3 The Four Pillars Visibility Mapping

OOP Principle	Class / Method in Code	Technical Structural Explanation
Abstraction	abstract class Donation public abstract double getValue()	Isolates high-level system logic from the specific mathematical details of different donation formulas.
Inheritance	MoneyDonation, FoodDonation, ClothingDonation	Concrete subclasses extend Donation to reuse common tracking parameters, eliminating code redundancy.
Polymorphism	Overridden toString() d.getTaxDeductibleAmount()	Enables the application loop to process diverse records using generic references, choosing the correct execution logic at runtime.
Encapsulation	private String donorName private ArrayList<Donation> donations	Hides sensitive tracking state behind strict access modifiers, forcing all mutations to pass through controlled public endpoints.

4. IMPLEMENTATION

4.1 Key Code Excerpts Description

Excerpt 1: Polymorphic Parent Structure & Sorting Interface (Donation.java)

```
public abstract class Donation implements Comparable<Donation>, TaxDeductible {
    private static int counter = 1;
    private final int id;
    private final String donorName;
    private final LocalDate date;

    public Donation(String donorName, LocalDate date) {
        this.id = counter++;
        this.donorName = donorName;
        this.date = date;
    }

    public int getId() { return id; }
    public String getDonorName() { return donorName; }
    public LocalDate getDate() { return date; }

    public abstract double getValue();

    @Override
    public int compareTo(Donation other) {
        return Double.compare(other.getValue(), this.getValue());
    }
}
```

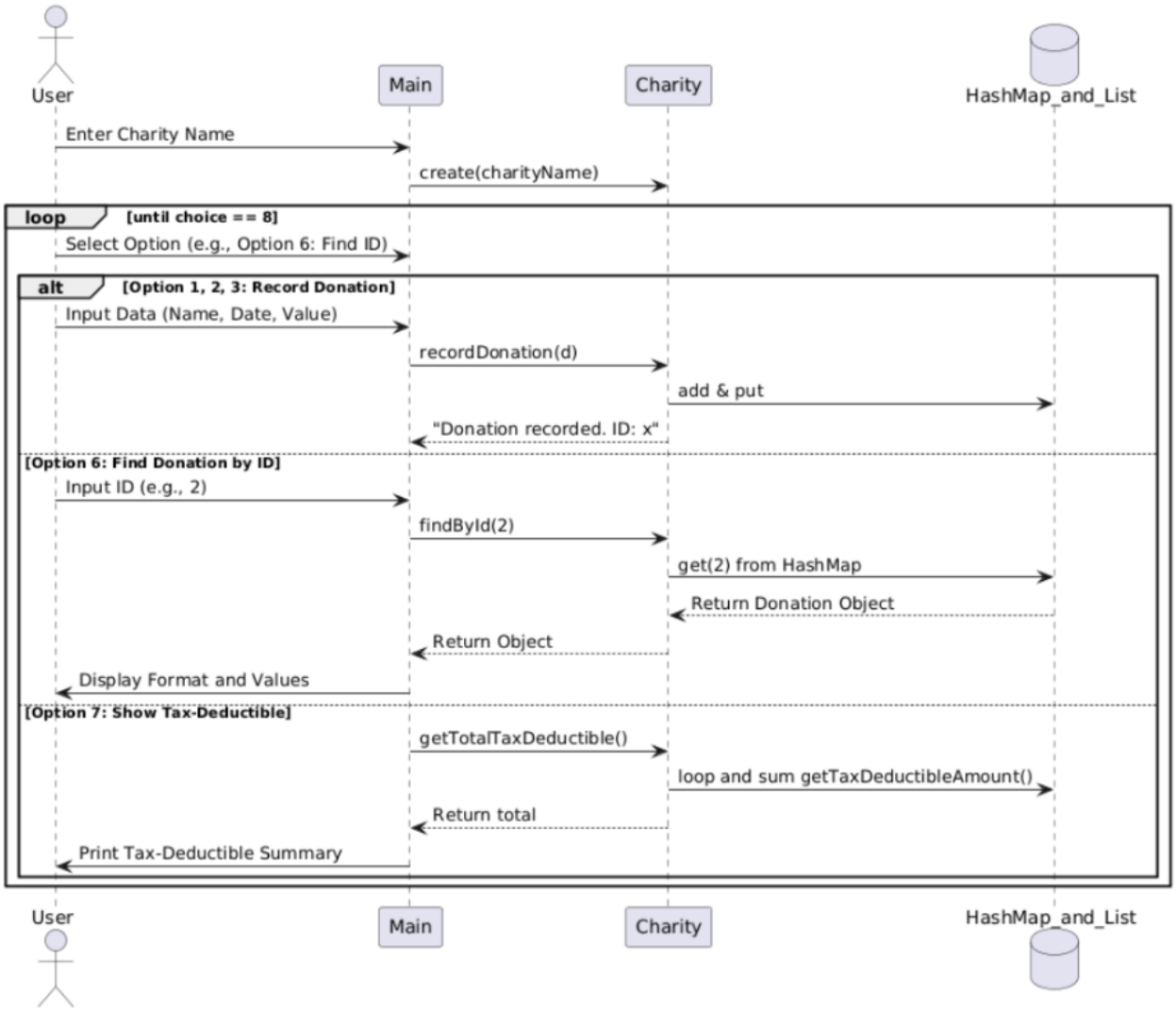
Excerpt 2: Synchronized Repository Storage Engine (Charity.java)

```
public class Charity {
    private final String charityName;
```

```
    private final ArrayList<Donation> donations = new ArrayList<>();
    private final HashMap<Integer, Donation> donationById = new HashMap<>();

    public Charity(String charityName) { this.charityName = charityName; }

    public void recordDonation(Donation d) {
        donations.add(d);
        donationById.put(d.getId(), d);
        System.out.println("Donation recorded. ID: " + d.getId());
    }

    public Donation findById(int id) {
        return donationById.get(id);
    }
}
```

5. TESTING AND RESULTS

5.1 System Execution Flow Diagram



5.2 Edge-Case Test Matrix

Test ID	Scenario Description	Mock Input String	Intended System	Result State

			Behavior	
TC-01	Happy-Path Standard Entry	1 \n Aslı \n 2025-12-01 \n 500	Registers instance; maps to ID 1; records value as 500.00 TL.	PASSED
TC-02	Rapid O(1) Target Lookup	6 \n 2	Instantly fetches record via key; correctly outputs parameters and calculations.	PASSED
TC-03	Missing Key Boundary Query	6 \n 99	Key not found; returns gracefully with No donation found with that ID.	PASSED
TC-04	Invalid Entry Format Shield	Date input: 2025/12/01	Intercepts formatting issue; skips entry without crashing the application loop.	PASSED
TC-05	Out-of-Bounds Menu Option	Selection: 15	Catches out-of-bounds menu	PASSED

			selection; logs Unknown option. Skipping... verbatim.	

5.3 Complete Verification Run Transcript

```
--- Charity Menu ---
1. Record a money donation
2. Record a food donation
3. Record a clothing donation
4. List all donations
5. List sorted by value
6. Find donation by ID
7. Show total tax-deductible amount
8. Exit
Choose an option (1-8): 1
Donor name: ASLI
Date (yyyy-mm-dd): 2001-03-06
Amount (TL): 5000
Donation recorded. ID: 1

--- Charity Menu ---
1. Record a money donation
2. Record a food donation
3. Record a clothing donation
4. List all donations
5. List sorted by value
6. Find donation by ID
7. Show total tax-deductible amount
8. Exit
Choose an option (1-8): 2
Donor name: BURAK
Date (yyyy-mm-dd): 2005-05-20
Weight (kg): 5
Donation recorded. ID: 2
```

```
--- Charity Menu ---
1. Record a money donation
2. Record a food donation
3. Record a clothing donation
4. List all donations
5. List sorted by value
6. Find donation by ID
7. Show total tax-deductible amount
8. Exit
Choose an option (1-8): 3
Donor name: SELEN
Date (yyyy-mm-dd): 2002-03-24
Item count: 6
Donation recorded. ID: 3

--- Charity Menu ---
1. Record a money donation
2. Record a food donation
3. Record a clothing donation
4. List all donations
5. List sorted by value
6. Find donation by ID
7. Show total tax-deductible amount
8. Exit
Choose an option (1-8): 4
ID: 1 | Donor: ASLI (2001-03-06) [Money] - 5000,00 TL | Value: 5000,00 TL
ID: 2 | Donor: BURAK (2005-05-20) [Food] - 5.0 kg | Value: 400,00 TL
ID: 3 | Donor: SELEN (2002-03-24) [Clothing] - 6 items | Value: 900,00 TL
```

```
--- Charity Menu ---
1. Record a money donation
2. Record a food donation
3. Record a clothing donation
4. List all donations
5. List sorted by value
6. Find donation by ID
7. Show total tax-deductible amount
8. Exit
Choose an option (1-8): 5
===== Sorted by Value (high -> low) =====
ID: 1 | Donor: ASLI (2001-03-06) [Money] - 5000,00 TL | Value: 5000,00 TL
ID: 3 | Donor: SELEN (2002-03-24) [Clothing] - 6 items | Value: 900,00 TL
ID: 2 | Donor: BURAK (2005-05-20) [Food] - 5.0 kg | Value: 400,00 TL

--- Charity Menu ---
1. Record a money donation
2. Record a food donation
3. Record a clothing donation
4. List all donations
5. List sorted by value
6. Find donation by ID
7. Show total tax-deductible amount
8. Exit
Choose an option (1-8): 6
Enter donation ID: 3
ID: 3 | Donor: SELEN (2002-03-24) [Clothing] - 6 items | Value: 900,00 TL
Value: 900,00 TL | Tax-Deductible: 450,00 TL
```

```
--- Charity Menu ---
1. Record a money donation
2. Record a food donation
3. Record a clothing donation
4. List all donations
5. List sorted by value
6. Find donation by ID
7. Show total tax-deductible amount
8. Exit
Choose an option (1-8): 7
===== Tax-Deductible Summary =====
Total Tax-Deductible Amount: 5690,00 TL
```

```
--- Charity Menu ---
1. Record a money donation
2. Record a food donation
3. Record a clothing donation
4. List all donations
5. List sorted by value
6. Find donation by ID
7. Show total tax-deductible amount
8. Exit
Choose an option (1-8): 15
Unknown option. Skipping...
```

```
--- Charity Menu ---
1. Record a money donation
2. Record a food donation
3. Record a clothing donation
4. List all donations
5. List sorted by value
6. Find donation by ID
7. Show total tax-deductible amount
8. Exit
Choose an option (1-8): 6
Enter donation ID: 2
ID: 2 | Donor: BURAK (2005-05-20) [Food] - 5.0 kg | Value: 400,00 TL
Value: 400,00 TL | Tax-Deductible: 240,00 TL

--- Charity Menu ---
1. Record a money donation
2. Record a food donation
3. Record a clothing donation
4. List all donations
5. List sorted by value
6. Find donation by ID
7. Show total tax-deductible amount
8. Exit
Choose an option (1-8): 6
Enter donation ID: 55
No donation found with that ID.
```

6. LIMITATIONS AND FUTURE WORK

Currently, all collected records are stored within temporary, volatile system memory (ArrayList and HashMap collections). As a result, turning off or restarting the application deletes all tracking history, making it unsuitable for long-term database use.

Future development cycles will focus on expanding this standalone prototype into a secure, multi-tier data platform. Planned features include adding data persistence via SQL or binary file streams, implementing input verification rules, applying encryption standards to protect donor data, and building an interactive JavaFX Graphical User Interface (GUI) to replace the command-line interface.

7. CONCLUSION

This project successfully demonstrates how core object-oriented concepts can streamline manual, unorganized non-profit workflows. By using inheritance to reuse code, separating logic cleanly via interfaces, and implementing a synchronized dual-storage manager, the team created a structured, efficient application. The development lifecycle emphasized the importance of defensive error shielding and optimized search strategies, providing our group with practical software engineering skills that are directly applicable to complex system design.

8. REFERENCES

[1] J. Bloch, Effective Java, 3rd ed. Boston, MA: Addison-Wesley, 2018.

[2] Java Standard Edition Platform Development Documentation, Oracle Corporation, 2025. [Online]. Available: https://docs.oracle.com/javase/8/docs/api/

[3] S. Çolakoğlu, "EE1004 Object-Oriented Programming Group Project Specification Guidelines," Marmara University, Istanbul, 2026.